LLM-Guided Iterative Fuzzing Process Prompt Template

Introduction of VFCS:

Provide a brief introduction to the Vulnerable Function Call Sequences (VFCS) and their significance in smart contract fuzzing. Explain how VFCS can help in systematically exploring vulnerabilities in contracts by guiding the fuzzing process.

Current Fuzzing Setup:

Please provide the following information:

- The source code of smart contract.
- Real-time feedback from fuzzing.

Feedback Integration:

Integrate the feedback and coverage data into the following template for analysis:

Feedback Information:

- **Total Branch Snippet (Possible Branches)**
- **Covered Branch Snippet (Current Branch Explanation)**
- **Branch Coverage**
- **Identified Vulnerability Type**
- **Current Fuzzing Time**
- **Current Sequence**

VFCS Generation Task:

Requirements: Analyze the provided information and predict what the next fuzzing VFCS should look like to achieve higher coverage.

Format of the generated VFCS:

```
mainContractName_1:
function1 -> function2 -> ...
```

Note: The generated VFCS will be used as input for fuzzing based on the last given VFCS. Hence, the currently covered places will guide the next fuzzing round. Focus should be on the branches that are not covered, generating new VFCS to target these uncovered paths.

Questions for Analysis:

1. Summarize the current contract state.
2. Analysis and Strategy:

- What uncovered branches exist? Identify them.
- Possible strategies to consider:
  - Repeated Calls: Allow functions to appear multiple times in the sequence to increase the chance of covering untested branches.
  - Explore Different Paths: Try different combinations that are more likely to connect to uncovered branches.
  - Explore New Function Calls: Consider adding new functions or different combinations of existing functions to ensure coverage of all logical flows.

3. Design a new VFCS to cover the uncovered branches (generate two possible VFCS sequences according to the given format).